

AUTIM

Sistema Educacional Inclusivo

Documentação Técnica dos Diagramas de Modelagem

Sprint 4 — Projeto Integrador

Curso: Análise e Desenvolvimento de Sistemas

Documento: Análise de Rastreabilidade e Consistência entre Requisitos e Diagramas

Versão consolidada — inclui revisão do Diagrama de Casos de Uso, Diagrama Conceitual e duas versões do Diagrama de Classes

1. Introdução e Contexto

Este documento consolida a documentação técnica das quatro versões revisadas dos diagramas de modelagem do sistema Autim, produzidas por um integrante do grupo a partir da primeira versão desenvolvida coletivamente. O objetivo é registrar, para cada diagrama, seu propósito, sua estrutura e seu funcionamento, além de indicar quais inconsistências identificadas na versão anterior foram corrigidas e quais ainda permanecem em aberto.

Os diagramas analisados são:

- Diagrama de Casos de Uso — visão de interação entre atores e funcionalidades do sistema.
- Diagrama Conceitual (Entidade-Relacionamento) — visão abstrata das entidades de negócio e seus relacionamentos.
- Diagrama de Classes — Versão A — reexportação estética da modelagem de classes original, em notação UML clássica.
- Diagrama de Classes — Versão B — remodelagem mais completa, com cobertura ampliada de métodos de CRUD e ciclo de vida das entidades.

A análise segue o mesmo critério de rastreabilidade aplicado anteriormente: cada elemento do diagrama é confrontado com os Requisitos Funcionais (RF), Regras de Negócio (RN) e Histórias de Usuário (US) do documento de Levantamento de Requisitos do projeto.

2. Diagrama de Casos de Uso

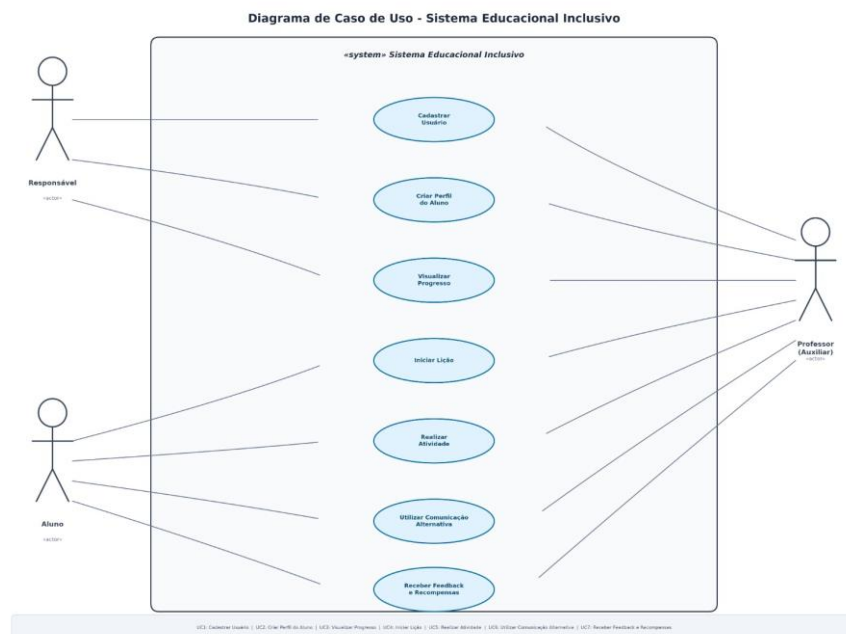


Figura 1 — Diagrama de Casos de Uso (versão revisada)

2.1 Contexto

Representa as interações entre os três atores do sistema e as sete funcionalidades principais, identificadas na legenda como UC1 a UC7, todas delimitadas dentro da fronteira do sistema («system» Sistema Educacional Inclusivo).

2.2 Atores e casos de uso associados

Ator	Casos de uso
Responsável	Cadastrar Usuário; Criar Perfil do Aluno; Visualizar Progresso
Aluno	Iniciar Lição; Realizar Atividade; Utilizar Comunicação Alternativa; Receber Feedback e Recompensas
Professor (Auxiliar)	Todos os 7 casos de uso (UC1 a UC7)

2.3 Correção identificada

✓Correção aplicada

Na versão anterior do diagrama, o ator era nomeado apenas "Professor", termo que não constava no documento de requisitos (que usa "Auxiliar" em RF01, RN03, US07, US08 e US09). Nesta versão, o ator foi renomeado para "Professor (Auxiliar)", eliminando a divergência de nomenclatura entre o diagrama e o levantamento de requisitos.

- MIDIA, FRASE, SIMBOLO_FRASE — comunicação alternativa por imagens e símbolos
- ROTINA, ITEM_ROTINA — rotinas diárias do aluno
- ATIVIDADE, SESSAO_ATIVIDADE — atividades educativas e execução das sessões
- RELATORIO — geração de relatórios de desempenho

3.3 Como funciona

Os relacionamentos utilizam os verbos POSSUI e TEM como entidades associativas, seguindo a notação de Peter Chen, com cardinalidades no formato (mínimo, máximo) em cada lado da relação — por exemplo, PERFIL —POSSUI— ALUNO (0,n) e ATIVIDADE —POSSUI— SESSAO_ATIVIDADE (0,n).

3.4 Pontos de atenção mantidos

⚠Pendência não resolvida

A estrutura geral deste diagrama é essencialmente a mesma da versão anterior — trata-se de um refinamento visual, não de uma revisão de modelagem. A cardinalidade do relacionamento ALUNO_USUARIO continua permitindo, na representação gráfica, que um aluno exista sem vínculo obrigatório a um responsável, o que diverge do texto da RN01 ("cada criança deve estar vinculada a pelo menos um responsável"). Recomenda-se ajustar a cardinalidade mínima para (1,1) ou (1,n) do lado do aluno, ou documentar explicitamente que essa obrigatoriedade é garantida por regra de aplicação.

4. Diagrama de Classes — Versão A

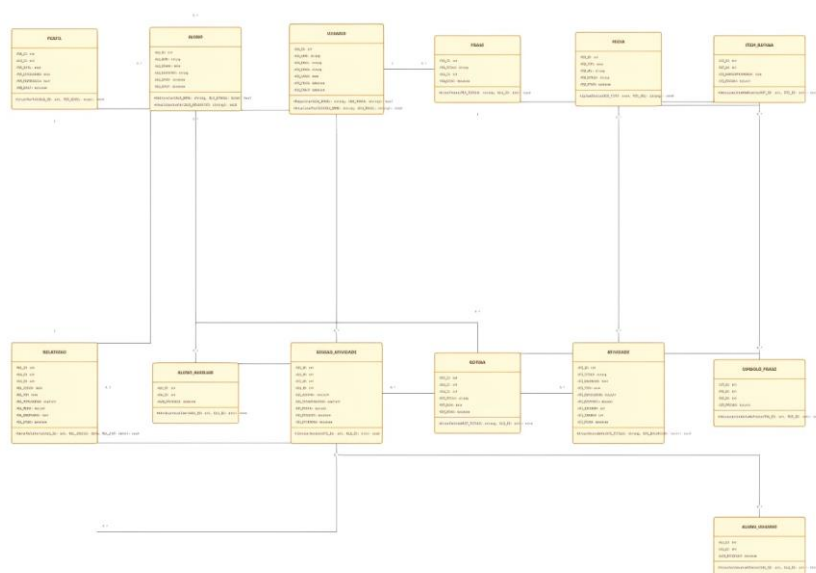


Figura 3 — Diagrama de Classes, notação UML clássica (Versão A)

4.1 Contexto

Reexportação em estilo UML clássico (retângulos com três compartimentos: nome da classe, atributos e métodos) da mesma modelagem de classes já existente — mantém os mesmos nomes de atributos com prefixo por tabela (ALU_, USU_, PER_, FRA_, entre outros) e os mesmos métodos já documentados anteriormente (Matricular, Registrar, CriarPerfil, entre outros).

4.2 Como funciona

Segue a notação UML padrão: cada classe apresenta atributos tipados e métodos com assinatura completa (parâmetros e tipo de retorno). As linhas entre classes representam associações, com a multiplicidade indicada nas extremidades (por exemplo, 0..* e 1..*).

4.3 Observação

i Nota

Esta versão é funcionalmente equivalente ao diagrama de classes analisado anteriormente. As lacunas já identificadas — ausência de método de autenticação (Login) e ausência de método de exportação de relatório — permanecem presentes nesta versão, pois trata-se de um reestilo visual, não de uma nova modelagem de comportamento.

5. Diagrama de Classes — Versão B

Esta é a versão mais relevante das quatro entregues, pois representa uma remodelagem de fato — e não apenas uma alteração estética — do diagrama de classes do sistema.

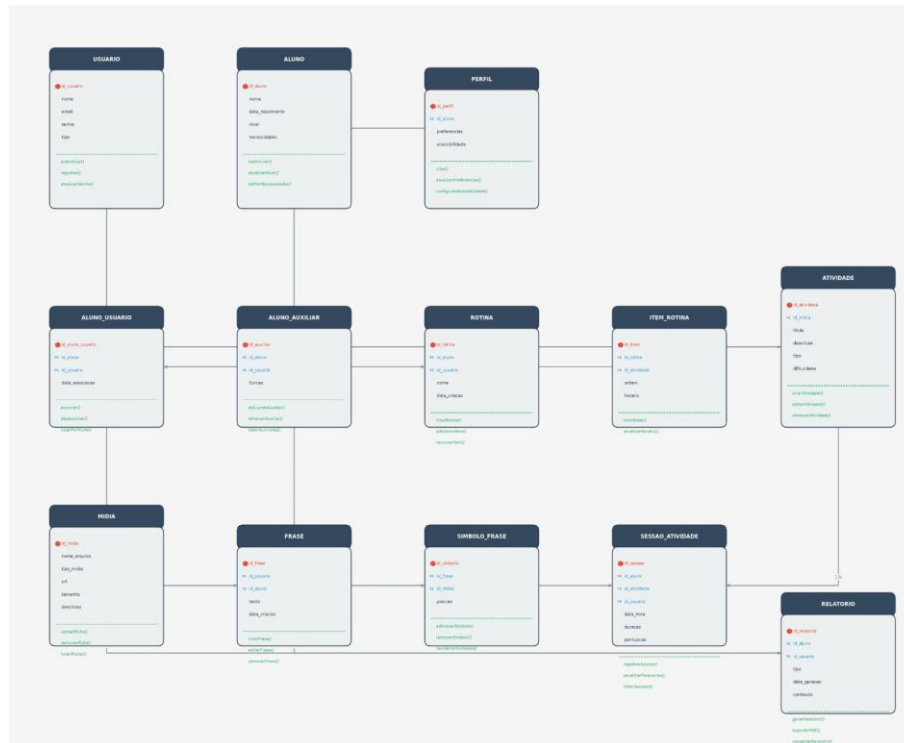


Figura 4 — Diagrama de Classes, versão ampliada com ciclo de vida completo (Versão B)

5.1 Contexto

Redesenho completo do diagrama de classes, com nomenclatura simplificada (sem prefixo de tabela — id_usuario, nome, email, em vez de USU_ID, USU_NOME) e cobertura muito mais ampla de métodos, contemplando o ciclo de vida completo de cada entidade: criação, atualização, remoção e listagem.

5.2 Estrutura de classes e responsabilidades

Classe	Atributos-chave	Métodos
USUARIO	id_usuario, nome, email, senha, tipo	autenticar(), registrar(), atualizarSenha()
ALUNO	id_aluno, nome, data_nascimento, nivel, necessidades	matricular(), atualizarNivel(), definirNecessidades()
PERFIL	id_perfil, id_aluno (FK), preferencias, acessibilidade	criar(), atualizarPreferencias(), configurarAcessibilidade()
ALUNO_USUARIO	id_aluno_usuario, id_aluno (FK), id_usuario (FK), data associacao	associar(), desassociar(), listarPorAluno()
ALUNO_AUXILIAR	id_auxiliar, id_aluno (FK), id_usuario (FK), funcao	adicionarAuxiliar(), removerAuxiliar(), listarAuxiliares()

Classe	Atributos-chave	Métodos
ROTINA	id_rotina, id_aluno (FK), id_usuario (FK), nome, data_criacao	criarRotina(), adicionarItem(), removerItem()
ITEM_ROTINA	id_item, id_rotina (FK), id_atividade (FK), ordem, horario	reordenar(), atualizarHorario()
ATIVIDADE	id_atividade, id_midia (FK), titulo, descricao, tipo, dificuldade	criarAtividade(), editarAtividade(), removerAtividade()
MIDIA	id_midia, nome_arquivo, tipo_midia, url, tamanho, descricao	uploadMidia(), removerMidia(), listarMidias()
FRASE	id_frase, id_usuario (FK), id_aluno (FK), texto, data_criacao	criarFrase(), editarFrase(), removerFrase()
SIMBOLO_FRASE	id_simbolo, id_frase (FK), id_midia (FK), posicao	adicionarSimbolo(), removerSimbolo(), reordenarSimbolos()
SESSAO_ATIVIDADE	id_sessao, id_aluno (FK), id_atividade (FK), id_usuario (FK), duracao, pontuacao	registrarSessao(), atualizarPontuacao(), listarSessoes()
RELATORIO	id_relatorio, id_aluno (FK), id_usuario (FK), tipo, data_geracao, conteudo	gerarRelatorio(), exportarPDF(), visualizarRelatorio()

5.3 Correções identificadas

✓Correção 1 — RF02 (Login)

O método autenticar() foi adicionado à classe USUARIO, resolvendo a ausência de representação do login/autenticação (RF02), que não constava em nenhum dos quatro diagramas anteriores (casos de uso, conceitual, lógico e classes).

✓Correção 2 — RF12 (Exportar relatório)

O método exportarPDF() foi adicionado à classe RELATORIO, resolvendo a ausência de suporte à exportação/compartilhamento de relatórios (RF12), que também estava ausente nos diagramas conceitual, lógico e de classes anteriores.

✓Correção 3 — RF11 (Gestão de rotina)

Os métodos editarAtividade(), removerAtividade(), reordenar() (em ITEM_ROTINA) e removerItem() (em ROTINA) resolvem a lacuna parcial de RF11/US06, que antes contava apenas com métodos de criação, sem cobrir edição ou remoção de itens da rotina.

5.4 Como funciona

As setas entre classes indicam navegabilidade e dependência — por exemplo, **MÍDIA** aponta para **FRASE** e para **ATIVIDADE**, indicando que essas classes consomem mídia como recurso. A notação de multiplicidade "1:N", visível próxima a **ATIVIDADE** e **RELATORIO**, reforça a cardinalidade um-para-muitos entre uma atividade e as sessões e relatórios gerados a partir dela.

6. Resumo Comparativo — Evolução dos Diagramas

A tabela a seguir consolida o status de cada inconsistência identificada na análise da versão anterior dos diagramas, frente às versões revisadas apresentadas neste documento.

Inconsistência identificada anteriormente	Status atual
RF02 (login) nunca modelado em nenhum diagrama	Resolvido — método <code>autenticar()</code> na classe USUARIO (Diagrama de Classes, Versão B)
RF12 (exportar relatório) nunca modelado	Resolvido — método <code>exportarPDF()</code> na classe RELATORIO (Diagrama de Classes, Versão B)
Inconsistência de nomenclatura "Professor" vs. "Auxiliar"	Resolvido — ator renomeado para "Professor (Auxiliar)" no Diagrama de Casos de Uso
Falta de métodos de edição/remoção em Rotina e Item de Rotina (RF11)	Resolvido — <code>removerItem()</code> , <code>reordenar()</code> e <code>atualizarHorario()</code> adicionados (Diagrama de Classes, Versão B)
RF04/RF05 (adaptação e recomendação automática) sem classe "Sistema" dedicada	Ainda ausente em todos os diagramas revisados
Cardinalidade de ALUNO_USUARIO permite aluno sem responsável (RN01)	Ainda não corrigida no Diagrama Conceitual

6.1 Considerações finais

A revisão realizada pelo integrante do grupo representa um avanço real na consistência entre os diagramas e o documento de requisitos, corrigindo quatro das seis pendências levantadas na análise anterior. O Diagrama de Classes — Versão B é claramente a modelagem mais madura entre as apresentadas e é recomendado como referência oficial do projeto, em substituição à Versão A.

Permanecem como pontos de atenção para as próximas sprints:

- Definir se a lógica de adaptação e recomendação automática (RF04, RF05) será modelada como uma classe/serviço "Sistema" ou tratada exclusivamente como lógica de aplicação sem persistência própria.
- Ajustar a cardinalidade mínima do relacionamento ALUNO_USUARIO no Diagrama Conceitual (e, por consequência, no Lógico) para refletir a obrigatoriedade definida na RN01.